Package/Method	Description	Code Example

Base64 response format

Instead of returning URLs, you can get images as base64 data for immediate use without downloading from a URL. Useful when you need to process or store the images directly.

```python
import base64
from PIL import Image
import io


response = client.images.generate(
    model="gpt-image-2",
    prompt="a white siamese cat",
    size="1024x1024",
    response_format="b64_json",  # Get base64 instead of URL
    n=1,
)
```

```javascript
// Convert base64 to image
image_data = base64.b64decode(response.data[0].b64_json)
image = Image.open(io.BytesIO(image_data))
image.show()  # Display the image
```

Credentials setup

Sets up the credentials for accessing the watsonx API. The api_key is not needed in the lab environment, and the project_id is preset.

```python
from ibm_watsonx_ai import Credentials
import os


credentials = Credentials(
    url="https://us-south.ml.cloud.ibm.com",
    )
```

```
project_id="skills-network"
```

GPT Image generation

Uses GPT Image to generate an image based on a text prompt. GPT Image supports generations, edits, and variations, simultaneously allowing up to 8 images.

```python
response = client.images.generate(
    model="gpt-image-2",
    prompt="a white siamese cat",
    size="1024x1024",
    quality="low",
    n=1,
)
```

Effective prompting

Tips for crafting effective prompts to get better results from image generation models:

- Be specific and detailed in your descriptions
- Include artistic style references
- Specify lighting, perspective, and composition
- Add context or setting information

```
// Basic prompt
prompt = "a cat"


// Improved detailed prompt
prompt = "a fluffy white siamese cat with
blue eyes sitting on a window sill,
golden hour lighting, soft shadows,
shallow depth of field,
professional photography style"
```

```
// Artistic style prompt
prompt = "a white siamese cat in the style
of a Renaissance oil painting, dramatic
lighting, rich colors, detailed fur texture"
```

File download

Function to download an image file from a URL if it doesn't already exist locally.

```python
import requests
```

```python
def load_file(filename, url):
    # Download file if it doesn't already exist
    if not os.path.isfile(filename):
        print("Downloading file")
        response = requests.get(url, stream=True)
        if response.status_code == 200:
            with open(filename, 'wb') as f:
                f.write(response.content)
        else:
            print("Failed to download file. Status code:", response.status_code)
    else:
        print("File already exists")
```

Image captioning

Loop through the images to see the text descriptions produced by the model in response to the query, "Describe the photo".

```python
user_query = "Describe the photo"
for i in range(len(encoded_images)):
    image = encoded_images[i]
    response = generate_model_response(image, user_query)
    // Print the response with a formatted description
    print(f"Description for image {i + 1}: {response}/n/n")
```

Image display

Displays an image in the notebook using IPython's display functionality.

```python
from IPython.display import Image
```

```
Image(filename=filename_tim, width=300)
```

Image encoding

Encodes an image to base64 format for inclusion in the model request. This is necessary because JSON is text-based and doesn't support binary data directly.

```python
import base64
import requests
```

```python
def encode_images_to_base64(image_urls):
    encoded_images = []
    for url in image_urls:
        response = requests.get(url)
        if response.status_code == 200:
            encoded_image = base64.b64encode(response.content).decode("utf-8")
            encoded_images.append(encoded_image)
            print(type(encoded_image))
        else:
            print(f"Warning: Failed to fetch image from {url} (Status code: {response.status_code})")
            encoded_images.append(None)
    return encoded_images
```

Message formatting

Creates a structured message containing both text and image data to send to the model.

```
messages = [{
    "role": "user",
    "content": [
      {
        "type": "text",
        "text": question
      },
      {
        "type": "image_url",
        "image_url": {
          "url": "" + encoded_string,
        }
      }
    ]
  }]
    return messages
```

Model invocation

Sends the formatted message to the model and receives a response with an analysis of the image.

```python
response = model.chat(messages=my_message_1)
print(response["choices"][0]["message"]["content"])
```

Model initialization

Initializes the vision model with specific parameters for text generation.

```python
from ibm_watsonx_ai.foundation_models.schema import TextChatParameters
from ibm_watsonx_ai.foundation_models import ModelInference


model_id = 'ibm/granite-vision-3-2-2b'


params = TextChatParameters(
    temperature=0.2,
    top_p=0.5,
)
```

```
model = ModelInference(
    model_id=model_id,
    credentials=credentials,
    project_id=project_id,
    params=params
)
```

Multiple images (GPT Image)

Generate multiple images at once with GPT Image using the 'n' parameter. GPT Image can generate up to 8 images in a single request.

```python
response = client.images.generate(
    model="gpt-image-2",
    prompt="a white siamese cat",
    size="1024x1024",
    quality="low",
    n=4,  # Generate 4 different images
)
```

```
// Access all generated images
for i, image_data in enumerate(response.data):
    image_bytes = base64.b64decode(image_data.b64_json)
    display.Image(data=image_bytes, width=256))
```

OpenAI client initialization

Creates an instance of the OpenAI client to interact with the API.

```python
from openai import OpenAI
from IPython import display
```

```
client = OpenAI()
```

Object dectection

Ask the model to define objects from a specific image.

```python
image = encoded_images[1]
user_query = "How many cars are in this image?"
print("User Query: ", user_query)
print("Model Response: ", generate_model_response(image, user_query))
```

pip install

Installs the necessary Python libraries required for working with watsonx and vision models.

```
%pip install ibm-watsonx-ai==1.1.20 image==1.5.33 requests==2.32.0
```

Quality options

Quality settings for generated images: low, medium, high

```
// GPT Image with high-definition quality
response = client.images.generate(
    model="gpt-image-2",
    prompt="a mountain landscape",
    size="1024x1024",
    quality="high",
    n=1,
)
```

Saving generated images

Save the generated images to your local filesystem for later use.

```
import requests


// Save from URL
response = client.images.generate(
    model="gpt-image-2",
    prompt="a white siamese cat",
    size="1024x1024",
)


image_bytes = base64.b64decode(response.data[0].b64_json)
with open("generated_cat.jpg", "wb") as f:
    f.write(image_bytes)
```

```
print("Image saved to generated_cat.jpg")
```

Size options

Different size options available for GPT Image models: 1024x1024, 1024x1536, 1536x1024, auto

```
// GPT Image with larger size and portrait mode orientation
response = client.images.generate(
    model="gpt-image-2",
    prompt="a white siamese cat",
    size="1024x1536",
    quality="low",
    n=1,
)
```

Author

Hailey Quach

Cheat Sheet: Integrating Visual and Video Modalities

The content of this lab is licensed under Apache 2.0

Package/Method	Description	Code Example
Base64 response format	Instead of returning URLs, you can get images as base64 data for immediate use without downloading from a URL. Useful when you need to process or store the images directly.	python `import base64` `from io import BytesIO` `import os` `response = client.images.generate(` `  model="gpt-image-1",` `  prompt="A cute baby sea otter",` `  n=1,` `  size="1024x1024",` `  response_format="b64_json",  # Get base64 instead of URL` `)` `# Decode and save the image` `image_base64 = response.data[0].b64_json` `image_bytes = base64.b64decode(image_base64)` `with open("otter.png", "wb") as f:` `  f.write(image_bytes)` `print("Image saved as otter.png")` shell `# Generate and decode in one step` `curl -s -X POST "https://api.openai.com/v1/images/generations" \ \` `  -H "Content-Type: application/json" \ \` `  -H "Authorization: Bearer $OPENAI_API_KEY" \ \` `  -d '{` `  "model": "gpt-image-1",` `  "prompt": "A cute baby sea otter",` `  "n": 1,` `  "size": "1024x1024",` `  "response_format": "b64_json"` ` }'	jq -r '.data[0].b64_json'	base64 --decode > otter.png` `echo "Image saved as otter.png"`

Credentials setup	Sets up the credentials for accessing the watsonx API. The api_key is not needed in the lab environment, and the project_id is saved.	python
		`from ibm_watsonx_ai import Credentials`
		`import os`
		
		`credentials = Credentials(`
		`  url="https://us-south.ml.cloud.ibm.com",`
		`  api_key="0000000000000000000000000000000000000000"`
		`  )`
		
		shell
		`project_id="skills-network"`

GPT-Image generation	Use GPT Image to generate or image-based a cover prompt. GPT Image supports generation, edits, and variations, while basically allowing up to 10 images.	python
		`response = client.images.generate(`
		`  model="gpt-image-1",`
		`  prompt="A sea otter cowboy cat",`
		`  n=1,`
		`  size="1024x1024",`
		`  quality="low",`
		`  )`
		`# ...`

Effective prompting	Tips for crafting effective prompts to get better results from image generation models:
	- Be specific and detailed in your descriptions
	- Include artistic style references
	- Specify lighting, composition, and color preferences
	- Add context or setting information
		python
		`# Detailed prompt`
		`prompt = """`
		
		`# Impressionist watercolor prompt`
		`prompt = """A misty forest clearing at dawn,`
		`Monet-style painting, oil on canvas, soft,`
		`blurry edges, pastel colors, dewdrops,`
		`shallow depth of field,`
		`professional dramatic lighting, style"""`

File download	Function to download an image file from a URL if it doesn't already exist locally.	python
		`import requests`
		
		copy
		`def download_file(filename, url):`
		`  """Download file if it doesn't already exist."""`
		`  if not os.path.isfile(filename):`
		`  print(f"Downloading {filename} from {url}...")`
		`  response = requests.get(url, stream=True)`
		`  if response.status_code == 200:`
		`  with open(filename, 'wb') as f:`
		`  for chunk in response.iter_content(1024):`
		`  f.write(chunk)`
		`  print(f"Downloaded {filename} successfully.")`
		`  else:`
		`  print(f"Failed to download {filename}. Status code: {response.status_code}")`
		`  else:`
		`  print(f"File {filename} already exists.")`

Image captioning	Loops through the images to use the text descriptions produced by the model in response to the query, "Describe the photo".	python
		`each_query = "Describe the photo"`
		`for i, x in enumerate(encoded_images):`
		`  print(f"Processing image {i+1}...")`
		`  response = generate_photo_caption(image, each_query)`
		`  # Print the response with a formatted explanation`
		`  print(f"Response from the image {i + 1}: {response}\n")`

Image display	Displays an image in the notebook using IPython's display functionality.	python
		`from IPython.display import Image`
		
		shell
		`Image(filename="otter.png")`

Image encoding	Encodes an image to base64 format for inclusion in the model request. This is necessary because JSON is text-based and doesn't support binary data directly.	python
		`import base64`
		`import requests`
		
		copy
		`def encode_image_to_base64(image_url):`
		`  """Encode image to base64 format."""`
		`  try:`
		`  if image_url.startswith(('http://', 'https://')):`
		`  # Download image from URL`
		`  response = requests.get(image_url)`
		`  response.raise_for_status()`
		`  image_data = response.content`
		`  else:`
		`  # Read local file`
		`  with open(image_url, 'rb') as image_file:`
		`  image_data = image_file.read()`
		`  return base64.b64encode(image_data).decode('utf-8')`
		`  except Exception as e:`
		`  print(f"Error encoding image {image_url}: {type(e).__name__}: {e}")`
		`  return None`

Message formatting	Creates a structured message containing both text and image data to send to the model.	json
		`messages = [`
		`  {`
		`  "role": "user",`
		`  "content": [`
		`  {`
		`  "type": "text",`
		`  "text": question`
		`  },`
		`  {`
		`  "type": "image_url",`
		`  "image_url": {`
		`  "url": f"{ encoded_string}",`
		`  }`
		`  }`
		`  ]`
		`  }`
		`]`
		`return messages`

Model Invocation	Sends the formatted message to the Anderbrot and receive a response with annotations of the image.	python
		`response = model.chat(messages=my_message_1)`
		`print(response["choices"][0]["message"]["content"])`

Model Initialization	Initializes the vision model with specific parameters for text generation.	python
		`from ibm_watsonx_ai.foundation_models.schema import TextChatParameters`
		`from ibm_watsonx_ai.foundation_models import ModelInference`
		
		`model_id = "meta-llama/llama-3-2-11b-vision-instruct"`
		
		`params = TextChatParameters(`
		`  temperature=0.5,`
		`  top_p=0.9,`
		`  )`
		
		shell
		`model = ModelInference(`
		`  model_id=model_id,`
		`  credentials=credentials,`
		`  params=params,`
		`  project_id="skills-network"`
		`)

Multiple images (GPT Image)	Generate multiple images at once with GPT Image using the "n" parameter. GPT Image can generate up to 10 images in a single request.	python `response = client.images.generate(` `  model="gpt-image-1",` `  prompt="A whimsical treehouse city",` `  n=3,  # Generate 3 different images` `  size="1024x1024",` `  quality="low",` `  )` `# ...  # Generate 3 different images` shell `# Generate and save multiple images` `curl -s -X POST "https://api.openai.com/v1/images/generations" \ \` `  -H "Content-Type: application/json" \ \` `  -H "Authorization: Bearer $OPENAI_API_KEY" \ \` `  -d '{` `  "model": "gpt-image-1",` `  "prompt": "A whimsical treehouse city",` `  "n": 3,` `  "size": "1024x1024",` `  "quality": "low"` ` }'	jq -r '.data[].b64_json'	\` ` while IFS= read -r b64; do` ` count=$((count + 1))` ` echo "$b64"	base64 --decode > "treehouse_${count}.png"` ` done`

Openai client initialization	Creates an instance of the OpenAI client to interact with the API.	python
		`from dotenv import load_dotenv`
		`from display import display`
		
		shell
		`client = load()`

Object detection	Ask the model to define objects from a specific image.	python
		`image = encoded_images[0]`
		`each_query = "How many cars are there in this image?"`
		`for i, x in enumerate(encoded_images):`
		`print(f"Model Response: {generate_photo_caption(image, each_query)}") `

pip install	Installs the necessary Python libraries required for working with watsonx and vision models.	shell `%pip install 'ibm-watsonx-ai>=1.3.39' langchain-ibm 'pillow>=10.4.0' \`

Quality options	Quality settings for generated images: low, medium, high	shell
		`# GPT Image with high-definition quality`
		`curl -s -X POST "https://api.openai.com/v1/images/generations" \ \`
		`  -H "Content-Type: application/json" \ \`
		`  -H "Authorization: Bearer $OPENAI_API_KEY" \ \`
		`  -d '{`
		`  "model": "gpt-image-1",`
		`  "prompt": "A whimsical landscape",`
		`  "n": 1,`
		`  "size": "1024x1024",`
		`  "quality": "high"`
		` }'
			jq -r '.data[0].b64_json'	base64 --decode > landscape_hd.png`

Saving generated images	Save the generated image to your local filesystem for later use.	python
		`import requests`
		
		`# Send POST request`
		`response = client.images.generate(`
		`  model="gpt-image-1",`
		`  prompt="A collage of four cute baby sea otters",`
		`  n=1,`
		`  size="1024x1024"`
		`  )`
		
		`image_bytes = base64.b64decode(response.data[0].b64_json)`
		`with open("otter_collage.png", "wb") as f:`
		`  f.write(image_bytes)`
		
		shell
		`curl "http://www.example.com/cat.png" \`

Size options	Different size options available for GPT Image models: 1024x1024, 1536x1024, 1024x1536, or auto.	shell
		`# GPT Image with larger size and portrait mode orientation`
		`response = client.images.generate(`
		`  model="gpt-image-1",`
		`  prompt="A tall redwood forest",`
		`  n=1,`
		`  size="1024x1536",`
		`  quality="low",`
		`  )`
		`# ...`

Author

Ashutosh Sharma

Skills Network

IBM

The content of this lab is licensed under Apache 2.0